OpenAgenet / OAN Yellow Paper: Technical Architecture for Trust-Governed Agent Identity and Discovery

Jinliang Xu
China Academy of Information and Communications Technology, Beijing, China
xujinliang@caict.ac.cn; jlxufly@gmail.com

***Abstract*—This yellow paper describes the technical architecture of OpenAgenet / OAN. OAN is a protocol-neutral trust layer for open Agent interconnection. It specifies the role architecture, identity objects, registration workflow, Root-governed lifecycle, Root-verified package model, authorization-aware Discovery, signed trusted invocation, verification requirements, state transitions, security properties, implementation boundaries, and deployment considerations. The design is intended to support heterogeneous Agent frameworks and interaction protocols, including MCP, A2A, ANP-like systems, and domain-specific Agent protocols. OAN does not define the entire business conversation among Agents; it defines how Agent identities become admissible, discoverable, verifiable, and safe to approach before protocol-specific interaction begins.**

Open-source project: https://github.com/OpenAgenet

***Index Terms*—OpenAgenet, OAN, Agent identity, DID, VC, discovery, signed envelope, trusted invocation**

1. Technical Scope

OAN specifies a trust-governed Agent identity and Discovery architecture. The system is concerned with the admissibility of Agent identity representations across registration, Root acceptance, package distribution, Discovery indexing, Discovery response verification, and pre-connection invocation. It deliberately does not define a complete task protocol, model orchestration framework, prompt strategy, tool execution API, ranking algorithm, or business ontology.

The technical scope can be expressed through the following invariant: an Agent identity should become visible to a relying party only after it has been prepared by the Agent operator, checked by an authorized Registrar, accepted by Root, distributed through a verifiable package, indexed by an authorized Discovery node, and returned with evidence that can be checked before invocation.

The current formal route style is intentionally unversioned:

- Root: `/root/...`
- Registrar: `/registrar/...`, `/agents/...`, `/capability-tree`
- Discovery: `/discovery/...`, `/discover/query`
- CDN: `/cdn/...`

Route versioning can be introduced later through content negotiation, typed object versions, or explicit API prefixes. The current design keeps the route surface simple while placing version information in protocol objects where verification actually occurs.

TABLE I
OAN Protocol Stack

Profile	Main objects	Primary users
Identity	DID document, metadata	Agent SDKs
Registration	draft, proof, credential	Registrar, Root
Governance	bulletin, authorization events	Root, official operators
Package	verified package, cursor	Root, CDN, Discovery
Discovery	query, signed response	Discovery, User Agent
Invocation	signed request/response	User/Service Agent
Adapter	MCP/A2A/ANP mapping	protocol integrators

2. OAN Protocol Stack

OAN should be read as a protocol stack rather than as one service API. The stack has several profiles, each of which can be implemented independently and combined by different operators. Its lower layers reuse the DID/VC family of identity and credential concepts [1], [2], [3], [4], while its upper layers are designed to wrap Agent interaction protocols such as MCP and A2A [5], [6].

The stack structure clarifies implementation scope. An Agent developer may only need the Identity, Discovery, and Invocation profiles. A Discovery operator needs Package and Discovery profiles plus Root bulletin verification. A full infrastructure operator needs Registration, Governance, Package, and Discovery. An adjacent protocol community can define an Adapter profile without changing the Root acceptance model.

3. System Assumptions

OAN assumes a trust domain with one Root authority. This does not mean that the future Agent Internet must be governed by one universal root. It means that the current architecture defines the internal mechanics of one trust domain first. Federation among multiple roots is treated as future work.

OAN also assumes that participants can perform standard cryptographic verification. Agents and infrastructure nodes are expected to hold DID-style identity material and signing keys. Signed objects use canonical serialization and hash binding. Time synchronization is assumed to be good enough for timestamp freshness windows, while nonce storage handles replay protection. The trust posture is aligned with zero-trust and workload-identity principles: every privileged role and runtime request must be explicitly verifiable rather than implicitly accepted by network location [7], [8].

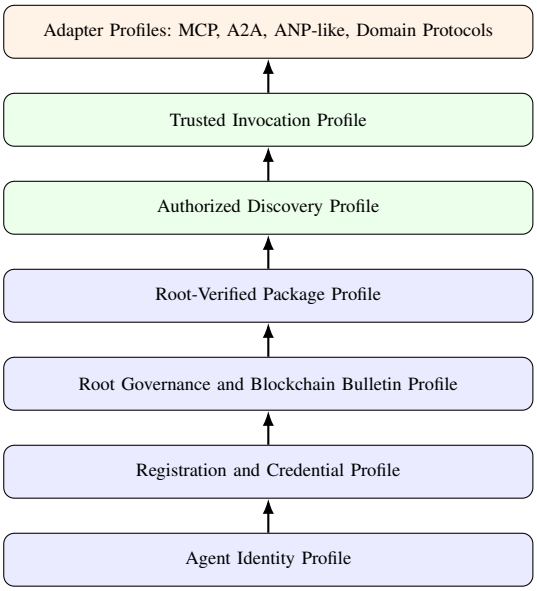

Fig. 1. OAN technical profiles form a layered trust stack.

TABLE II
OAN TRUST LAYERS

Layer	Purpose	Mechanism
Governance review	admission policy	operator process
Credential issuance	attest review result	VC-like credential
Subject control	prove DID ownership	challenge signature
Root acceptance	admit identity version	signed Root record
Package distribution	publish identity state	Root proof and hash
Discovery exposure	scoped query visibility	domain authorization
Runtime invocation	verify request peer	signed envelope

The protocol-facing trust path should not depend on private review evidence such as email, phone, web account login, or manually reviewed forms. Such evidence may support governance workflow, but runtime verification should rely on DID documents, credentials, signed envelopes, Root bulletin facts, Root-verified packages, and Discovery response proofs.

4. TRUST LAYERS

OAN separates social governance, admission workflow, and runtime protocol verification. The layers are shown in Table II.

This separation makes failures easier to reason about. If a Registrar is misconfigured, Root still checks Registrar authorization and credential proof. If a CDN returns a tampered package, Discovery recomputes hashes and verifies Root proof. If a Discovery node is stale or unauthorized, User Agents can reject responses that fail freshness or authorization checks. If a caller replays an old invocation, the Service Agent can reject it using nonce and timestamp validation.

5. CORE ENTITIES AND NOTATION

Let $\mathcal{A}$ be Agent subjects, $\mathcal{R}$ Registrar nodes, $\mathcal{D}$ Discovery nodes, and ρ the Root authority in one trust domain. An Agent $a \in \mathcal{A}$ has a versioned identity representation M_a^v. The representation hash is $H(M_a^v)$. A Root-accepted record contains at least:

$$\langle did_a, v, H(M_a^v), status, t, \sigma_\rho \rangle.$$

The record binds Agent DID, version, representation hash, lifecycle status, timestamp or logical time, and Root signature. The exact serialization may evolve, but the binding requirement is stable.

An identity representation is admissible to a Discovery node d only if it is well formed, anchored by Root, current, and authorized for d:

$$\begin{aligned} Admissible(d, M) = {} & WellFormed(M) \wedge Anchored(M) \\ & \wedge Current(M) \wedge Scope(d, M). \end{aligned}$$

This predicate separates business search from trust admission. A query may match many metadata fields, but only admissible candidates should be exposed.

6. ROLE ARCHITECTURE

The technical roles are intentionally separated.

A. Root

Root owns governance state, accepted-version state, capability-domain governance, bulletin facts, and verified package generation. It verifies Registrar authorization, signed upstream envelopes, registration credentials, DID structures, document hashes, capability metadata, and status transitions. Root should be the source of truth for current accepted Agent identity versions.

Root is not the business data plane. It should not need to observe every Agent-to-Agent call. This reduces operational load and avoids turning the trust anchor into a universal traffic proxy.

In the next-stage design, Root also acts as the official governance operation entrance for infrastructure lifecycle actions. It coordinates and validates operator intent, while a blockchain-backed bulletin publishes lifecycle events for Registrar, Discovery, and future VC issuer nodes.

B. Registrar

Registrar owns onboarding assistance, draft workflow, capability-tag suggestion, subject-control challenge handling, registration credential issuance, and Root submission. It is an operational gateway between Agent operators and Root. Registrar can improve usability and review quality, but it does not unilaterally decide final Root acceptance.

C. Discovery

Discovery owns package synchronization, Root proof verification, bulletin checking, authorization-domain filtering, local indexing, query handling, and response signing. Discovery is allowed to optimize search and ranking, but its first obligation is to avoid exposing non-admissible identity representations. In the blockchain-backed bulletin model, Discovery consumes authorization events and maintains a local authorization cache; it does not need to embed proposal or voting logic in the default service runtime.

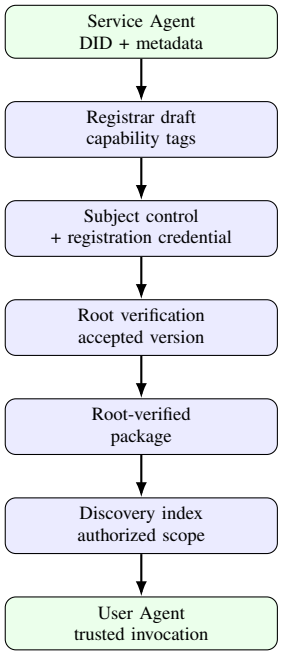

Fig. 2. OAN identity lifecycle from Agent preparation to trusted invocation.

D. CDN

CDN owns distribution of Root-verified packages and manifests. CDN improves availability and scalability but is not a trust authority. Consumers should verify package hashes, Root proof, metadata, and bulletin consistency.

E. Service Agent and User Agent

The Service Agent is the callable business Agent. The User Agent is the relying client Agent. Their trust relationship begins with Discovery verification and continues through signed invocation. OAN does not dictate the complete application conversation after the trust guard passes.

7. Protocol Objects

A. DID-Style Identity Representation

The identity representation is compatible with DID Document concepts. It contains identifier, verification methods, service endpoints, capability tags, credential references, and OAN metadata. OAN does not require a single fixed DID method, although the reference implementation uses `did:ans`.

The identity representation should include enough material to support:

- key discovery and signature verification;
- endpoint discovery for supported Agent protocols;
- capability-domain matching;
- credential binding;
- version and status tracking;
- Root package generation and verification.

B. Capability Metadata

Capability metadata has a canonical component and an extensible component. The canonical component maps to a Root-governed capability tree. The extensible component allows application-specific tags, search hints, labels, or descriptions. Only canonical capability domains should expand Discovery authorization. Custom tags may refine search within an authorized set, but they must not grant exposure outside authorized domains.

C. Registration Credential

The Agent registration credential binds:

- issuer Registrar DID;
- subject Agent DID;
- credential type and claims;
- registration binding, including DID-control evidence;
- issuance time, validity, and proof.

Root verifies issuer authorization, credential proof, subject binding, credential status, and validity before accepting a submitted representation.

D. Subject-Control Challenge

Subject-control proof prevents a Registrar from issuing a credential for a DID that the applicant cannot control. The challenge should bind draft identifier, subject DID, Registrar DID, expiration time, and nonce. The Agent signs the challenge using an assertion-capable verification method from its DID document. Registrar verifies the signature and stores evidence or a verifiable summary.

E. Signed Upstream Envelope

Privileged infrastructure writes, especially Registrar-to-Root submissions, use a signed upstream request envelope. The envelope covers sender DID, target audience, method, path, body hash, timestamp, nonce, purpose, and protocol version. Root verifies the sender's current authorization and checks replay state before processing the body.

F. Root-Verified Package

The package consumed by CDN and Discovery includes the DID Document, metadata, hashes, Root proof, publication cursor, subject version, status, and capability information. Discovery does not trust the CDN path alone; it recomputes hashes and verifies Root proof and bulletin consistency.

G. Discovery Response

Discovery responses are signed by the Discovery node and include returned candidates, provenance metadata, timestamp, query context, and proof. A relying User Agent can verify that the response came from a currently authorized Discovery node and that selected candidates trace back to Root-verified packages.

8. Blockchain-Backed Bulletin

The next-stage OAN governance profile can use a blockchain-backed bulletin as the event source for infrastructure-node lifecycle state. Related SSI systems have also explored blockchain-backed trust anchors and lifecycle coordination [9], [10], [11]. The contract scope is intentionally narrow. It manages lifecycle authorization for Registrar, Discovery, and future third-party VC issuer nodes. It does not manage Agent registration records, full identity documents, capability-tree updates, Root key rotation, or CDN service coordination.

A. Lifecycle Scope

The bulletin records and emits events for:

- Registrar authorization, suspension, recovery, and revocation;
- Discovery authorization, authorized-domain updates, suspension, recovery, and revocation;
- third-party VC issuer authorization and revocation.

The chain event stream replaces repeated status checks against Root for these infrastructure lifecycle facts. Registrar, Discovery, and VC issuer services subscribe to or poll bulletin events, rebuild local authorization state, and enforce the resulting state in their own service logic.

B. Governance Operation Boundary

In the current version, governance transactions are mainly performed by official operation clients, such as a Root operator console, governance command line tool, or governance service. These clients create proposals, submit votes, refresh proposal status, and manage committee membership.

Registrar, Discovery, and VC issuer service nodes are event consumers by default. Their runtime should include event subscription or polling and local authorization-cache maintenance, but it should not need proposal, vote, or member-management code. This separation keeps service nodes simpler and keeps governance private-key handling inside official governance tooling until the ecosystem is ready to admit broader community operators.

C. DID and VC Boundary

The bulletin contract does not store, parse, validate, or issue DID Documents or Verifiable Credentials. It also does not model the internal schema of `did:ans`, Agent identifiers, credential claims, credential proofs, Agent metadata, or capability descriptions.

Fields such as subject identifier, policy hash, metadata hash, effective time, expiration time, and status are only lifecycle references. DID and VC semantics remain off-chain OAN protocol responsibilities implemented by Root, Registrar, Discovery, VC issuer services, and SDK verification logic.

TABLE III
Blockchain Bulletin Boundary

In contract scope	Outside contract scope
Registrar lifecycle	Agent DID Document storage
Discovery lifecycle	VC issuance or validation
VC issuer lifecycle	Agent metadata parsing
Discovery domain updates	Capability tree governance
Lifecycle events	Root key and CDN coordination

TABLE IV
Core Object Responsibilities

Object	Producer	Consumer	Purpose
Identity doc	Agent	Registrar, Root	subject keys
Reg credential	Registrar	Root	onboarding proof
Bulletin event	gov. client	infra nodes	lifecycle fact
Verified package	Root	CDN, Discovery	accepted version
Discovery response	Discovery	User Agent	query provenance
Invocation envelope	User Agent	Service Agent	request binding
Signed response	Service Agent	User Agent	response origin

D. Trusted Invocation Envelope

Before invoking a Service Agent, the User Agent sends caller DID material, credentials, nonce, timestamp, target DID, request body hash, discovery provenance, and request proof. The Service Agent checks caller identity, credential subject, target binding, body hash, signature, nonce uniqueness, and timestamp freshness.

Table IV shows the design discipline of OAN: each object has a producer, a verifier, and a trust purpose. Objects should not be overloaded. For example, a Discovery response does not create Root acceptance; it only presents signed query provenance over already verified packages. A registration credential does not make an Agent discoverable; it only supports Root admission.

9. Lifecycle State Machine

OAN treats identity as stateful. A typical Agent identity moves through the states shown in Table V.

Create and update should use the same full-document path. Root should verify the complete new representation rather than applying ambiguous partial patches. This makes current-version verification easier and reduces risk that old fields remain accidentally authoritative.

10. Lifecycle Flow

The standard OAN lifecycle is:

1) Agent operator prepares a DID-style identity representation.
2) Registrar assists draft creation and capability-tag selection.
3) Agent proves control of its DID key.
4) Registrar issues an Agent registration credential.
5) Registrar submits the complete representation to Root with a signed upstream envelope.
6) Root verifies Registrar authorization, envelope proof, credential proof, subject binding, DID structure, and capability metadata.

TABLE V
AGENT IDENTITY LIFECYCLE STATES

State	Owner	Meaning
draft	Registrar	identity under preparation
control-verified	Registrar	DID control proof passed
credentialed	Registrar	registration credential issued
submitted	Registrar/Root	Root submission in progress
accepted	Root	version accepted by Root
published	Root/CDN	package ready for sync
indexed	Discovery	admissible for query
superseded	Root	replaced by newer version
revoked	Root	no longer admissible

7) Root archives the accepted version and creates a Root-verified package.
8) CDN distributes the package.
9) Discovery synchronizes packages, verifies Root proof and bulletin facts, applies authorized-domain filtering, and indexes eligible Agents.
10) User Agent queries Discovery and verifies the signed response.
11) User Agent invokes Service Agent with a trusted invocation envelope.
12) Service Agent verifies the envelope and returns a signed response.

11. ROOT VERIFICATION REQUIREMENTS

Root verification should be deterministic and auditable. At minimum, Root should verify:

- signed upstream envelope integrity;
- sender Registrar DID and authorization status;
- timestamp freshness and nonce uniqueness;
- request body hash binding;
- registration credential proof and issuer binding;
- subject DID binding between credential and document;
- DID document structure and verification methods;
- capability tags against the governed capability tree;
- version and status transition validity.

Rejection should be explicit. Silent acceptance with partial checks is more dangerous than a visible rejection because downstream Discovery and relying Agents may assume Root acceptance means the whole trust path was verified.

12. DISCOVERY AUTHORIZATION

OAN uses capability domains to control which Discovery node may expose which Agent identities. Let G be the governed capability tree and $Auth(d)$ be the authorized domain set for Discovery node d. If $T(M)$ is the set of canonical capability tags in identity representation M, then:

$$Scope(d, M) = \begin{cases} true, & T(M) \cap Desc(Auth(d)) \neq \emptyset, \\ false, & otherwise. \end{cases}$$

Custom tags are allowed, but they cannot expand the governance domain. They are used only after coarse authorization for local refinement, ranking, or search.

A. Extension Safety Rules

Extensions are useful for local search quality and domain integration, but they must not weaken authorization. The following rules should hold for custom tags, plugin metadata, semantic embeddings, ranking features, and domain-specific attributes:

- an extension must not cause Discovery to expose an identity outside the Discovery node's authorized canonical domains;
- an extension must not override Root acceptance, current-version state, revocation state, or package verification results;
- ranking and semantic matching must run only after admissibility checks;
- unknown non-critical extension fields may be ignored, while unknown critical fields must cause rejection by verifiers that do not support them;
- plugin output should be auditable enough for an operator to explain why an identity was indexed, rejected, ranked, or returned.

B. Synchronization Rules

Discovery synchronization should be incremental. A publication cursor or watermark allows Discovery to fetch new packages without repeatedly scanning the entire Root archive. Discovery should verify each package before indexing and should track rejected packages with reasons. A package that fails Root proof, hash, bulletin, status, or authorization checks should not enter the query index.

C. Query Rules

Discovery query can use keywords, capability tags, endpoint metadata, local indexes, or ranking signals. However, query matching should never bypass the admissibility predicate. The proper order is: verify package, check current state, enforce Discovery authorization, index admissible representation, then apply query matching and ranking.

13. TRUSTED INVOCATION

Trusted invocation is the bridge from discovery to protocol-specific interaction. OAN does not define every message that Agents exchange after the first verified request. It defines the guard that lets a Service Agent decide whether a caller and request deserve entry into the business protocol.

A Service Agent should verify:

- caller DID document and verification method;
- caller credential proof and subject binding;
- Discovery response proof and freshness, when included;
- selected-service provenance against Root-verified package data;
- target DID equals the invoked Service Agent DID;
- timestamp is inside an allowed freshness window;
- nonce has not been used before;
- body hash matches the received body;
- request signature verifies over the canonical envelope.

If these checks pass, OAN hands control to the application protocol. That protocol may be A2A, MCP-adjacent tool invocation, a domain API, or a custom workflow.

14. Security Properties

OAN targets the following properties:

- **Provenance**: accepted identity representations trace to a Root-authorized acceptance event.
- **Integrity**: tampered identity representations fail hash or proof verification.
- **Freshness**: stale, revoked, or superseded versions are not admissible after synchronization.
- **Authorization soundness**: compliant Discovery nodes do not index packages outside their authorized capability domains.
- **Replay resistance**: signed upstream writes and invocation requests bind timestamp, nonce, target, path, and body hash.
- **Role accountability**: each privileged decision is attributable to an authorized infrastructure identity.

15. Threat Model

A. Malicious or Unauthorized Registrar

An unauthorized Registrar may attempt to submit an identity to Root. Root rejects it because the signed upstream envelope signer is not active in Root authorization state. A malicious but authorized Registrar may issue poor credentials; OAN can detect invalid signatures, wrong subjects, and policy violations, but semantic review quality remains a governance problem.

B. Tampered Package Distribution

An attacker controlling a CDN path may return modified documents or metadata. Discovery recomputes hashes and verifies Root proof. Tampering should cause package rejection.

C. Overbroad Discovery Exposure

A Discovery node may try to expose packages outside its authorized domains. Compliant Discovery software should filter such packages before indexing. Relying parties can also verify Discovery authorization state. Non-compliant Discovery behavior remains auditable if returned candidates include provenance that contradicts authorization policy.

D. Stale Lifecycle Cache

A Registrar, Discovery, or VC issuer node may miss bulletin events because of network failure or local indexer downtime. The service should expose sync lag, last processed event sequence, and cache health. Authorization-sensitive actions should fail closed when lifecycle state is unavailable or too stale.

E. Replay and Wrong-Target Invocation

An attacker may replay a previously signed request or redirect a signed request to another Service Agent. Timestamp windows, nonce stores, target DID binding, path binding, and body hash binding reduce these risks.

F. Semantic Misrepresentation

An Agent may claim a capability that it does not perform well. OAN can bind and govern the identity claim, but it cannot fully prove business quality or truth of all semantic claims. Reputation, testing, certification, and runtime policy must complement OAN.

16. Compatibility with MCP, A2A, and ANP

OAN is not a replacement for MCP, A2A, ANP, or other Agent protocols [5], [6], [12].

MCP can use OAN to verify the identity and authorization context of tool servers or clients before tool access. An MCP client may check that a server endpoint belongs to a Root-accepted Agent identity. An MCP server may use OAN credentials to decide whether a caller is allowed to access a tool.

A2A can use OAN to verify Agent identity and discovery provenance before task exchange. An A2A Agent Card or capability description can be mapped into the OAN DID-style identity representation, while A2A remains responsible for task state and message semantics.

ANP-like routes and Agent DID methods can be mapped to OAN identity documents and lifecycle checks. OAN's distinctive contribution is not only having an Agent identifier; it is the governed identity lifecycle, Discovery authorization, package verification, and pre-connection trust guard around that identifier.

A. MCP Mapping

An MCP server can be represented as an OAN Service Agent or as a service endpoint inside an OAN identity representation. MCP tool metadata can be mapped to OAN service metadata and capability tags. Before MCP initialization or tool execution, a client can query OAN Discovery, verify the Discovery response, verify the Root package, and then bind the MCP endpoint to the verified Agent identity. MCP remains responsible for tool listing, tool invocation, resource access, and streaming behavior.

B. A2A Mapping

An A2A Agent Card can be mapped to an OAN identity representation by binding Agent identifier, service endpoints, capability declarations, and verification methods. OAN Discovery can serve as the pre-session discovery layer. After the candidate is verified, A2A can take over task negotiation, message flow, task state, and long-running collaboration. This avoids forcing A2A to solve Root governance and Discovery authorization internally.

C. ANP and DID Method Mapping

ANP-like route structures and Agent DID methods can be used as identifier and networking substrates. OAN can consume such identifiers if they can be resolved into verification methods and service metadata. The OAN-specific part is the registration path, Root acceptance, verified package, and authorized Discovery semantics around that identifier.

TABLE VI
PROTOCOL INTEGRATION BOUNDARIES

Protocol	Keeps responsibility for	OAN provides
MCP	tools and resources	peer identity trust
A2A	task exchange	trusted discovery
ANP	Agent networking	lifecycle governance
DID/VC	primitives	Agent-specific policy
Domain API	business action	pre-connection guard

TABLE VII
IMPLEMENTATION PROFILES

Profile	Required capabilities	Typical implementer
Agent	verify Discovery, sign calls	Agent runtime
Discovery	sync packages, sign query	directory operator
Registrar	draft, credential, submit	onboarding operator
Root	governance, accept, package	trust-domain operator
Adapter	map OAN to MCP/A2A/ANP	protocol integrator
Full stack	all infrastructure roles	trial network

17. IMPLEMENTATION PROFILES

OAN implementations do not need to implement every role at once. The following profiles define expected implementation depth.

The profile model is important for adoption. A partner can begin with an Agent profile, later operate a Discovery node, and only eventually operate Registrar or Root infrastructure. This makes OAN deployable as a layered ecosystem rather than an all-or-nothing platform.

18. IMPLEMENTATION BOUNDARY

The multi-repository implementation separates responsibilities:

- `oan-protocol-common`: protocol objects, DID/VC helpers, crypto abstraction, package model, bulletin model, and clients.
- `oan-reference-services`: Root, Registrar, Discovery, and CDN Rust services.
- `oan-agent-py`: Python Service Agent and User Agent demos.
- `oan-examples`: integration tests, demos, negative cases, and benchmarks.
- `oan-sdk-ts`: TypeScript SDK for web, Node.js, console, and partner integrations.
- `oan-adapters`: protocol adapters for MCP, A2A, ANP-like, and domain-specific protocol integration.
- `oan-discovery-plugins`: Discovery ranking, semantic matching, and domain plugin extensions.
- `oan-deploy`: deployment profiles and orchestration assets.
- `oan-web-console`: operator-facing infrastructure console.
- `oan-operator-guides`: runbooks, operating procedures, and cooperation guidance.
- `oan-site`: static website and browser-side reference UI.
- `oan-trial-network`: public trial-network records.
- `oan-release-tools`: signed release and artifact verification tooling.
- `oan-design-docs`: long-form design and governance rationale.

This repository split is a technical design choice. It keeps protocol objects, runtime services, examples, deployment, and governance records from drifting into private incompatible variants.

A. Normative Boundary

The technical design distinguishes normative protocol requirements from reference implementation choices. Normative requirements include signature coverage, hash binding, nonce and timestamp validation, Root proof verification, current-version checks, Discovery authorization filtering, failure reason reporting, and fail-closed behavior for security-critical verification. An implementation that omits these checks is not implementing the OAN trust model, even if it exposes similar HTTP routes.

Reference implementation choices include concrete database engines, queue implementations, local cache layouts, index structures, deployment scripts, dashboard design, and default retry policies. These choices may vary across operators as long as the externally visible verification semantics and conformance results remain consistent.

19. PERSISTENCE AND OPERATIONAL STATE

Root, Registrar, Discovery, and CDN each maintain state with different criticality.

- Root state is authoritative for authorization, accepted versions, publication, and bulletin facts.
- Registrar state is authoritative for draft workflow, subject-control evidence, credential issuance, and submission attempts.
- Discovery state is authoritative for local synchronization cursor, package verification results, rejected package reasons, and query indexes.
- CDN state is authoritative for distributed package availability and manifest history, but not for trust decisions.

SQLite is useful for the reference implementation and local experiments. Production deployment should support database abstraction and PostgreSQL or equivalent backends. Root should usually be migrated last because its state is the most central and has the highest consistency requirements. CDN and Discovery are natural earlier candidates for validating backend abstraction.

20. PERFORMANCE ENVELOPE

The prototype evaluation has covered single-node scales up to 2000 identity representations and logical multi-node scales up to 1200 total identity representations. The reported results show correct lifecycle execution, negative-case rejection, authorized Discovery completeness, and internal degraded-baseline effects. The main current bottlenecks are large-result query processing, synchronization, publication, and submission pressure at higher scales.

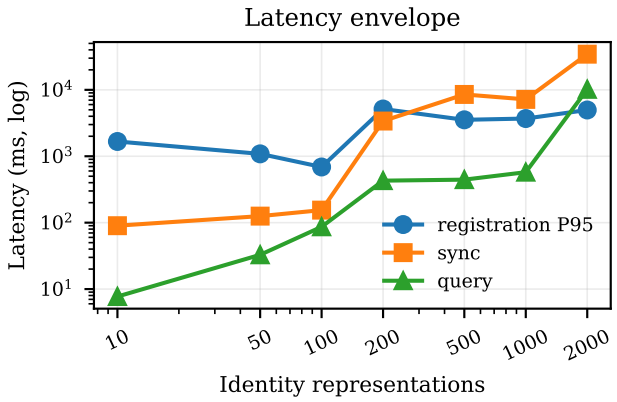

Fig. 3. Latency envelope across the single-node scalability experiment. The 2000-identity run exposes large-result query and synchronization pressure, which motivates pagination, index tuning, and queue hardening.

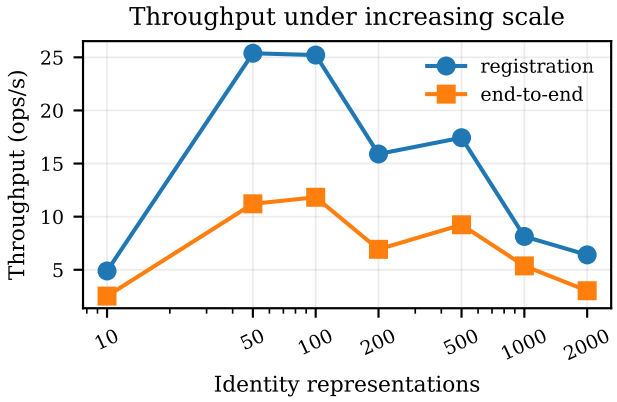

Fig. 4. Registration and end-to-end throughput under increasing identity scale. Throughput remains measurable through the tested range, while the gap between registration and end-to-end throughput highlights downstream publication, synchronization, and query costs.

The performance target of this yellow paper is not to claim final Internet-scale capacity. It is to define the technical architecture and expose where production hardening should focus: persistent queue tuning, query result pagination, index optimization, distributed deployment, observability, database backend evolution, and more precise backpressure.

The ablation experiment was executed on copied reference-service workspaces so that the normal implementation remained unchanged. Each degraded variant removed one mechanism and then exercised the corresponding Root-node or Discovery-node code path over the same generated cases. The full path produced zero false acceptances or false exposures across the tested invalid cases, whereas the degraded variants exposed all cases designed to target the removed check. These results support the claim that OAN's authorization, credential, and freshness checks are functional trust mechanisms rather than merely descriptive metadata.

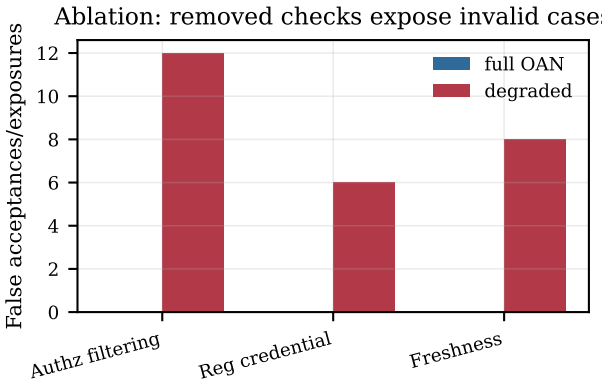

Fig. 5. Real-system degraded-baseline ablation. Removing authorization filtering, registration-credential verification, or freshness checks causes false exposure or false acceptance in the degraded variants, while the full OAN path rejects all tested invalid cases.

21. Conformance Expectations

Independent implementations should be able to pass conformance tests for:

- DID-style identity document parsing and canonical hashing;
- registration credential proof verification;
- subject-control challenge verification;
- signed upstream envelope verification;
- Root package verification;
- Discovery response signature verification;
- capability-domain authorization;
- nonce and timestamp replay rejection;
- current-version and revocation handling.

Conformance should include negative cases. A system that only succeeds on the happy path but accepts malformed credentials, stale packages, unauthorized Discovery exposure, or replayed requests is not OAN-compatible in the trust sense.

22. API Semantics

This yellow paper does not freeze every HTTP route or JSON field, but it does define the semantic contract that APIs should preserve.

A. Root APIs

Root APIs can be divided into management, verification, publication, and read interfaces. Management interfaces update infrastructure authorization state, capability-domain state, and operational configuration. Verification interfaces accept signed Registrar submissions and return deterministic accept or reject results. Publication interfaces create or expose Root-verified packages and publication cursors. Read interfaces expose status, bulletin facts, accepted versions, and authorized infrastructure state.

Root write APIs should require signed upstream envelopes. Root read APIs may be public or restricted depending on deployment, but the returned trust facts should be verifiable or derived from verifiable Root state.

TABLE VIII
REPRESENTATIVE ERROR CATEGORIES

Category	Example	Expected behavior
syntax	malformed object	reject before trust checks
crypto	bad signature	reject and log signer
authz	inactive Registrar	reject as unauthorized
freshness	stale timestamp	reject as expired
replay	repeated nonce	reject and audit
binding	wrong subject DID	reject as mismatch
scope	out-of-domain package	do not index
state	superseded version	hide from query

B. Registrar APIs

Registrar APIs support draft creation, capability tree retrieval, capability tag suggestion, subject-control challenge generation, proof submission, credential issuance, and Root submission. A Registrar API should preserve the relationship between a draft, its DID document hash, its subject-control proof, its credential, and its Root submission result.

Registrar should also expose enough status for an Agent operator to understand why a draft cannot progress. For example, missing capability tags, failed DID control, expired challenge, credential issuance failure, and Root rejection are different errors and should not be collapsed into one generic failure.

C. Discovery APIs

Discovery APIs support status, synchronization, package inspection, rejected package inspection, and query. Query responses should be signed and should include enough provenance for a User Agent to verify selected candidates. Discovery should not expose packages that have not passed package verification and authorization filtering.

D. CDN APIs

CDN APIs expose manifests, packages, documents, metadata, and publication history. CDN responses should be treated as distribution data, not trust decisions. Consumers should validate Root proof and hashes after fetching CDN content.

23. ERROR TAXONOMY

Precise errors are part of interoperability. Table VIII summarizes common error categories.

Errors should be safe to expose. External responses may avoid leaking sensitive operator details, but internal logs should preserve enough information for audit and debugging.

24. CONSISTENCY MODEL

OAN does not require every component to become globally consistent instantaneously. The architecture uses a staged consistency model.

A. Root Consistency

Root is authoritative for accepted identity versions and infrastructure authorization. Within one Root deployment, state transitions should be transactional enough to avoid accepting a version without a corresponding archive and publication state. Root should also avoid ambiguous current-version state for the same Agent DID.

B. Publication Consistency

Publication from Root to CDN and Discovery can be asynchronous. A newly accepted Agent may not be immediately queryable. This is acceptable if the system exposes publication cursors, synchronization status, and retry behavior. The important property is monotonic progress without silently losing accepted packages.

C. Discovery Consistency

Discovery is eventually consistent with Root publication. It should never compensate for staleness by accepting unverifiable packages. If it has not yet synchronized a new accepted version, it may return no result or an older current result depending on Root state and cursor timing, but it should not return a package that fails current-version checks once it has learned the superseding state.

25. KEY MANAGEMENT CONSIDERATIONS

Every privileged role depends on signing keys. Key management is therefore a first-class operational issue.

A. Root Keys

Root signing keys anchor the trust domain. They should be protected with the strongest operational controls in the deployment. Rotation must be planned so that relying parties can distinguish old valid Root records from forged records. A production design should publish key identifiers, validity windows, and rotation records.

B. Registrar and Discovery Keys

Registrar keys sign upstream submissions and credentials. Discovery keys sign query responses. Both roles should support revocation and rotation. Root authorization state should refer to current infrastructure DIDs and verification methods, so that deactivated keys stop being trusted.

C. Agent Keys

Agent keys prove subject control and sign runtime requests or responses. Agent key rotation should be handled as an identity update. Root should accept the new identity version only after verifying the complete updated representation and supporting evidence.

26. Canonicalization and Hashing

OAN relies on hash binding. This makes canonicalization important. If two implementations serialize the same logical object differently, signatures and hashes may fail. Protocol objects should therefore define canonical JSON serialization or another deterministic representation. The following objects especially require stable canonicalization:

- DID-style identity representation;
- registration credential unsigned payload;
- signed upstream request envelope;
- Root-verified package metadata;
- Discovery response payload;
- trusted invocation envelope.

Test vectors should include canonical byte strings and expected hashes. This will reduce the risk that independent SDKs become incompatible despite using the same field names.

27. SDK Architecture

SDKs are necessary because OAN asks application developers to perform several security-sensitive checks. A good SDK should expose high-level functions while still allowing operators to inspect verification details.

A. Agent SDK

The Agent SDK should help Service Agents and User Agents create signed envelopes, verify peer DID documents, verify credentials, check timestamps and nonces, verify Discovery responses, and produce signed responses. It should not hide verification failures behind vague exceptions.

B. Discovery SDK

The Discovery SDK should help clients query Discovery nodes and verify signed responses. It should return both candidate data and verification status. A developer should be able to distinguish "no candidates matched" from "Discovery response failed verification".

C. Infrastructure SDK

The Infrastructure SDK should support Root, Registrar, Discovery, and CDN administration. It should build signed upstream requests, verify Root packages, interact with capability trees, and support conformance tests. Governance transaction support for the blockchain-backed bulletin should be isolated in official governance tooling or a dedicated governance client. Default Registrar, Discovery, and VC issuer runtimes should depend on event subscription and authorization-cache APIs rather than proposal or voting APIs.

28. Deployment Topologies

A. Local Development

Local development can run one Root, one Registrar, one Discovery node, one CDN, one Service Agent, and one User Agent on a single machine. This topology is useful for reproducing lifecycle tests, negative cases, and small benchmarks.

B. Single-Domain Pilot

A single-domain pilot may separate Root, Registrar, Discovery, and CDN into different services or machines while keeping one operator. This topology tests network behavior, synchronization, package distribution, and operational observability.

C. Multi-Operator Trial

A multi-operator trial introduces independently operated Registrars and Discovery nodes. Root authorization state becomes more important, and capability-domain boundaries should be tested carefully. This topology is the first meaningful step toward ecosystem deployment.

D. Production Trust Domain

A production trust domain should add durable databases, key management systems, rate limits, monitoring, backup, disaster recovery, release signing, and operator procedures. It should also define who can change Root authorization state and how emergency revocation is performed.

29. Observability

Observability should be designed around lifecycle and trust events rather than only HTTP status codes. Operators should track:

- Registrar draft state transitions;
- subject-control challenge success and failure;
- credential issuance counts and failures;
- Root acceptance and rejection reasons;
- Root publication queue depth and latency;
- CDN package availability and manifest history;
- Discovery synchronization cursor lag;
- Discovery package rejection reasons;
- query latency, result counts, and pagination behavior;
- nonce replay rejection and timestamp failures.

These metrics help distinguish performance bottlenecks from trust failures. For example, a missing Discovery result may be caused by query mismatch, publication lag, out-of-domain authorization, failed package verification, or Root rejection. Observability should make these causes separable.

30. Formal Invariants

The following invariants summarize the technical design.

A. I1: Root Acceptance Integrity

If Root accepts an identity version, then the accepted record must bind the Agent DID, version, representation hash, status, and Root proof. Any later package claiming that acceptance must match this bound hash.

B. I2: Registrar Authorization

If Root accepts a Registrar-submitted identity, then the submitting Registrar must have been authorized at submission time and the signed upstream envelope must verify under the Registrar identity.

C. I3: Discovery Scope

If a compliant Discovery node indexes an Agent identity, then at least one canonical capability tag in the identity must fall within the Discovery node's authorized domain set or its descendants.

D. I4: Current-Version Visibility

If an Agent identity version has been superseded or revoked and Discovery has processed the relevant Root state, then that version must not remain visible as the current admissible version.

E. I5: Invocation Binding

If a Service Agent accepts a trusted invocation envelope, then the envelope signature, timestamp, nonce, target DID, body hash, and caller credential binding must have passed verification.

31. Backward and Forward Compatibility

OAN should evolve without breaking every implementation. Compatibility can be managed through typed objects, optional fields, protocol version fields, capability negotiation, and conformance profiles. Critical verification fields should be mandatory. Experimental metadata should be optional and must not change trust semantics unless a new profile explicitly says so.

A useful compatibility rule is: old verifiers may ignore unknown non-critical metadata, but they must reject objects that require unknown critical trust semantics. This prevents silent downgrade when new security-relevant fields are introduced.

A. Wire-Format Stability

Future protocol specifications should freeze the wire-level details that are only described semantically in this paper. These include canonical JSON rules, required and optional fields, field naming, object version identifiers, signature preimage construction, hash algorithms, HTTP method and path versions, pagination tokens, timestamp formats, nonce scope, error-code registry, and critical-extension markers.

Until those details are frozen, independent implementations should treat the reference objects and test vectors as the compatibility source of truth. A breaking field change should create a new object version or conformance profile. A non-breaking extension should be optional, ignored by old verifiers, and prohibited from changing trust decisions unless it is explicitly marked as critical and supported by the verifier.

32. Reference Verification Algorithms

The following algorithms describe expected verifier behavior at a high level. They are not tied to one programming language.

A. Root Submission Verification

Root submission verification should proceed in a fail-closed order. First, parse the request and reject malformed objects. Second, verify the upstream envelope signature and body hash. Third, check timestamp freshness and nonce uniqueness. Fourth, resolve the Registrar identity and confirm active authorization. Fifth, verify the registration credential, including issuer, subject, validity, and proof. Sixth, verify DID document structure, verification methods, capability metadata, and subject binding. Seventh, check version transition rules. Only after these checks should Root create an accepted record.

B. Discovery Package Verification

Discovery package verification should parse the package, recompute hashes, verify Root proof, check bulletin evidence, check current-version state, and apply capability-domain authorization. Only packages that pass all checks should enter the query index. Failed packages should be retained in a rejected-package store with reason codes so that operators can debug synchronization and policy issues.

C. User Agent Candidate Verification

A User Agent should verify the Discovery response signature, timestamp, Discovery authorization state, selected candidate provenance, Root package hashes, and current-version status before constructing an invocation. The User Agent should not treat Discovery output as plain search output. It is signed trust evidence that still requires validation.

D. Service Agent Invocation Verification

A Service Agent should verify caller identity, caller credential, target DID, request path, body hash, timestamp, nonce, and request proof. It should also apply local authorization policy after OAN checks pass. OAN proves who the caller claims to be and whether the request is fresh and bound; local policy decides whether that caller may perform the requested business action.

33. Pagination, Batching, and Backpressure

The reference experiments show that large-result queries, publication, and synchronization are important bottleneck areas. The architecture should therefore treat pagination, batching, and backpressure as protocol-relevant engineering concerns.

A. Pagination

Discovery query responses should support pagination when result sets become large. Pagination metadata must be covered by the Discovery response signature so that a client can verify page boundaries and query context. A page token should not allow a client to bypass authorization checks or retrieve stale results outside the original query policy.

B. Publication Batching

Root-to-CDN and Root-to-Discovery publication can be batched. Batch records should include cursor ranges, package identifiers, status, retry count, and error state. Batching should improve throughput without hiding individual package verification failures.

C. Backpressure

Registrar submission paths should have explicit backpressure. A burst of draft submissions should not create unbounded Root requests or local contention. Backpressure can include bounded queues, single-flight protection per draft, timeouts, retry policies, and clear submission-state reporting.

34. Degraded Behavior

Production systems must define behavior during partial failure.

A. Root Unavailable

If Root is unavailable, Registrar may continue draft preparation and local DID control checks, but it cannot claim Root acceptance. Discovery should continue serving previously verified current data within freshness policy, but it should not invent new Root facts.

B. CDN Unavailable

If CDN is unavailable, Discovery synchronization may lag. Existing verified index data may remain usable within policy, but new packages will not become visible until synchronization resumes. Operators should expose cursor lag and sync failures.

C. Discovery Stale

If Discovery is stale, User Agents may see incomplete results. This is safer than accepting unverifiable data. Discovery should publish status that allows clients and operators to understand synchronization lag.

D. Nonce Store Failure

If a Service Agent cannot check nonce uniqueness, it should fail closed for high-risk operations. For low-risk demonstrations, a degraded mode may exist, but it should be explicit and should not be presented as full trusted invocation.

35. Data Retention

OAN deployments should define retention policies for drafts, credentials, accepted records, rejected submissions, package history, Discovery rejection logs, query logs, and invocation evidence. Retention should balance audit requirements with privacy and operational cost.

Root accepted records and revocation history usually require longer retention because they define trust-domain history. Registrar drafts and private review evidence may need shorter or policy-specific retention. Discovery query logs may contain sensitive interest patterns and should be minimized or protected. Invocation logs should be controlled by the Service Agent's business domain and privacy obligations.

36. Release and Supply-Chain Integrity

The OAN software supply chain should be treated as part of the trust model. Signed releases, reproducible build metadata, dependency review, artifact hashes, and compatibility statements help operators know which implementation they are running. Release tooling should distinguish protocol-compatible changes from experimental or breaking changes.

Reference services should publish version information and protocol object support. SDKs should publish test-vector compatibility. Trial networks should record which service versions are active so that experiments and demos can be reproduced.

37. Production Readiness Checklist

Production deployment requires more than passing the reference integration tests. Operators should define and test at least the following controls:

- durable nonce stores and timestamp skew policy for privileged writes and trusted invocation;
- event synchronization lag monitoring for Registrar, Discovery, and VC issuer authorization state;
- fail-closed authorization cache behavior when bulletin or Root state is unavailable or too stale;
- package verification failure alerts and rejected-package retention;
- Discovery index rebuild, cursor recovery, and query-result pagination;
- bounded queues, backpressure, retry policy, and timeout policy for Registrar submission and Root publication paths;
- key rotation, key compromise response, and operator access-control procedures;
- audit log retention for Root decisions, Registrar submissions, Discovery synchronization, and trusted invocation failures;
- release verification, dependency review, and conformance-test records for deployed service and SDK versions.

These controls do not change the protocol model, but they determine whether an OAN deployment can be trusted operationally by other organizations.

38. Test Vectors and Profiles

Interoperability requires more than prose. OAN implementations should publish test vectors that allow another implementation to verify whether it computes the same hashes, signatures, and authorization decisions.

A. Core Test Vectors

Core test vectors should include a canonical DID-style identity representation, its canonical byte sequence, its expected hash, a valid registration credential, an invalid credential with a wrong subject, a valid upstream envelope, an envelope with a mismatched body hash, a Root-verified package, a Discovery response, and a trusted invocation envelope. Each vector should include the expected verification result and the reason for rejection when applicable.

B. Conformance Profiles

OAN can define conformance profiles for different implementation depths. A basic identity profile may only parse and verify identity documents and Root packages. A Discovery profile must additionally enforce capability-domain authorization and sign responses. A trusted invocation profile must verify caller credentials, nonces, timestamps, target binding, body hashes, and request signatures. A full infrastructure profile must implement Root, Registrar, Discovery, and CDN semantics.

C. Security Configuration Baseline

A production profile should define minimum security configuration. Examples include nonce retention windows, timestamp skew tolerance, maximum request body hash age, key rotation procedure, rejected-submission logging, Discovery sync lag alerts, package verification failure alerts, and disabled degraded modes for high-risk operations. These values may vary by deployment, but the presence of explicit configuration is itself part of operational maturity.

D. Interoperability Reports

Independent implementations should be able to publish interoperability reports. Such reports should state which profiles are supported, which test vectors passed, which optional extensions are implemented, and which known limitations remain. This creates a clearer ecosystem than informal claims of compatibility.

39. Privacy and Data Minimization

OAN identity representations are intended to be discoverable, but that does not mean every piece of operator evidence should be public. Private review evidence should remain in governance workflow or Registrar records unless there is a clear reason to publish it. Public packages should contain the material needed for verification and discovery, not unnecessary personal or operational data.

Future privacy work may include confidential queries, selective disclosure, private capability matching, scoped credentials, and policies for sensitive Agent categories. The current design leaves room for these extensions by separating Root acceptance, package publication, and Discovery exposure.

40. Open Technical Work

Important next steps include:

- multi-root federation and cross-domain trust negotiation;
- richer semantic Discovery and capability taxonomy governance;
- privacy-preserving Discovery and confidential query processing;
- durable nonce stores and stronger replay windows;
- formal conformance tests for independent implementations;
- production deployment profiles and operational security baselines;
- MCP and A2A adapters with OAN pre-connection verification;
- stronger observability for Root, Registrar, Discovery, and CDN;
- package pagination and query-result pagination;
- signed release tooling and reproducible artifact verification.

41. Conclusion

OAN defines a technical architecture for governed Agent identity and trusted Discovery. Its contribution is the coupling of Root-anchored provenance, current-version freshness, authorization-scoped Discovery, verified package distribution, and signed pre-connection validation. This coupling allows heterogeneous Agent protocols and implementations to interoperate on top of a shared trust substrate without forcing them into one task protocol or one Agent runtime.

Acknowledgment

The author thanks Jian Jin, Xie Jiagui, Li Bingqi, and Zhu Runkai for their support.

References

[1] W3C Credentials Community Group, "Decentralized identifiers (dids) v1.0," https://www.w3.org/TR/did-core/, 2022.

[2] W3C Verifiable Credentials Working Group, "Verifiable credentials data model v2.0," https://www.w3.org/TR/vc-data-model-2.0/, 2025.

[3] ——, "Verifiable credential data integrity 1.0," https://www.w3.org/TR/vc-data-integrity/, 2024.

[4] W3C Credentials Community Group, "Did resolution," https://w3c-ccg.github.io/did-resolution/, 2024.

[5] Model Context Protocol Contributors, "Model context protocol specification," https://modelcontextprotocol.io/specification/, 2025.

[6] A2A Protocol Contributors, "Agent2agent protocol specification," https://a2a-protocol.org/latest/specification/, 2025.

[7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST Special Publication 800-207, 2020.

[8] Cloud Native Computing Foundation, "The spiffe and spire project for workload identity," https://spiffe.io/, 2022.

[9] A. Grüner, A. Mühle, and C. Meinel, "Analyzing and comparing the security of self-sovereign identity management systems through threat modeling," in *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 2023, pp. 389–396.

[10] M. Glöckler, J. Sedlmeir, V. Schlatt, and N. Urbach, "A systematic review of identity and access management requirements in enterprises and potential contributions of self-sovereign identity," *Business & Information Systems Engineering*, vol. 65, no. 4, pp. 421–440, 2023.

[11] M. Popa, X. Yue, A. Boudguiga, A. Chibani, and H. Bécha, "Chaindiscipline: A blockchain-based self-sovereign identity framework for iot," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2571–2584, 2023.

[12] Agent Network Protocol Contributors, "Did:wba method specification: Web-based agent decentralized identifier," https://agentnetworkprotocol.com/en/specs/03-did-wba-method-specification/, 2025.

[13] J. Xu, "Grail: A deep-granularity hybrid resonance framework for real-time agent discovery via slm-enhanced indexing," *arXiv preprint arXiv:2605.02489*, 2026.

[14] J. Xu and B. Li, "Darwinnet: An evolutionary network architecture for agent-driven protocol synthesis," *arXiv preprint arXiv:2604.01236*, 2026.

[15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[16] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[17] D. B. Terry, M. M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer, and C. H. Hauser, "Managing update conflicts in bayou, a weakly connected replicated storage system," *Proceedings of SOSP*, pp. 172–183, 1995.

[18] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and T. Polk, "Internet x.509 public key infrastructure certificate and certificate revocation list (crl) profile," RFC 5280, 2008.

[19] M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X.509 internet public key infrastructure online certificate status protocol - ocsp," RFC 2560, 1999.

[20] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "ToolEmu: Identifying the risks of lm agents with an lm-emulated sandbox," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=NB5FP1A2yC